41039 Programming 1

Revision of Week 7: lists and generics



Dr. Xuan-Bach Le
le.xuan.bach@hcmut.edu.vn

Faculty of Computer Science and Engineering, HCMUT

September Session 2026

Today's revision: 14 questions, about 50 minutes

#	Type	Topic
Q1	Fill in the blanks	Indexes, nodes, type parameters
Q2	Multiple choice	Which loop removes every negative?
Q3	Multiple choice	Which line does not compile?
Q4	Predict the output	Insert, set and remove (Java)
Q5	Predict the output	A chain of nodes (Python)
Q6	Find the bug	The mark book (Java, 4 bugs)
Q7	Find the bug	The basket (Python, 3 bugs)
Q8	Write the code	The last few, for any type (Java)
Q9	Write the code	A help-desk queue (Python)
Q10	Write the code	A queue of linked nodes (Java)
Q11	Predict the output	Splice the nodes (Java)
Q12	Predict the output	Who holds the new head? (Python)
Q13	Find the bug	The linked queue (Java, 3 bugs)
Q14	Find the bug	The chain (Python, 3 bugs)

How it works

1. Read the question and answer it **alone** first. Write your answer down.
2. Then compare with a neighbour.
3. The answer appears on the **next click**.

Week 7 in five ideas (read this first)

1. **A list grows.** Indexes: 0 to `size()`−1. `add(i,x)` *inserts* (later elements shift right); `set(i,x)` *replaces*. Python: `insert`, `xs[i] = x`.
2. **Index or value?** `remove(1)` removes **index** 1; `remove(Integer.valueOf(1))` removes the **value** 1. Python: `pop(i)`, `remove(x)`.
3. **Removing while looping.** A removal shifts later elements left, so a forward loop skips one. Loop backwards, use `Iterator.remove()`, or build a new list.
4. **Nodes and queues.** A node holds a value and a link, `next`. A queue is first in, first out: Java `LinkedList`, Python deque.
5. **Generics.** In `class Box<T>`, `T` is the *type parameter*; in `Box<String>`, `String` is the *type argument*: a class, so `Integer`, not `int`.

```
List<Integer> xs = new ArrayList<Integer>();
xs.add(7);                          // boxed
xs.add(0, 4);                       // [4, 7]
xs.remove(Integer.valueOf(7));      // [4]
int first = xs.get(0);              // unboxed
```

Q1 Fill in the blanks Indexes, nodes and type parameters

Fill each blank with a word from the bank.

Each word is used at most once. Four words are not used.

- An `ArrayList`'s spare room is its **(1)** ; `size()` counts only the elements stored.
- `set(i, x)` **(2)** the element at index `i`; the size stays the same.
- On a `List<Integer>`, `remove(1)` treats 1 as an **(3)** .
- A linked list is a chain of **(4)** , each with a value and a link.
- In `class Box<T>`, `T` is a type **(5)** , filled in later, e.g. `Box<String>`.
- Java turns an `int` into an `Integer` for you: **(6)** .

Word bank (2 min, alone)

a) capacity	**f)** autoboxing
b) inserts	**g)** nodes
c) argument	**h)** replaces
d) index	**i)** size
e) parameter	**j)** iterator

Q1 Fill in the blanks Indexes, nodes and type parameters

Fill each blank with a word from the bank.

Each word is used at most once. Four words are not used.

- An `ArrayList`'s spare room is its **(1)**; `size()` counts only the elements stored.
- `set(i, x)` **(2)** the element at index `i`; the size stays the same.
- On a `List<Integer>`, `remove(1)` treats 1 as an **(3)**.
- A linked list is a chain of **(4)**, each with a value and a link.
- In `class Box<T>`, `T` is a type **(5)**, filled in later, e.g. `Box<String>`.
- Java turns an `int` into an `Integer` for you: **(6)**.

Answers

(1) a) capacity **(2)** h) replaces
(3) d) index **(4)** g) nodes
(5) e) parameter
(6) f) autoboxing

Not used: b) inserts, c) argument, i) size, j) iterator.

`inserts` cannot be (2): inserting makes the list longer. `String` in `Box<String>` is the type **argument**; `T` is the **parameter** it fills.

Q2 Multiple choice Which loop removes every negative?

Goal: remove **every** negative number from the `List<Integer> xs`, which starts as `[4, -1, -2, 5]`.

Version 1

```
for (int i = 0;
     i < xs.size(); i++)
    if (xs.get(i) < 0)
        xs.remove(i);
```

Version 2

```
for (int i = xs.size() - 1;
     i >= 0; i--)
    if (xs.get(i) < 0)
        xs.remove(i);
```

Version 3

```
Iterator<Integer> it
    = xs.iterator();
while (it.hasNext())
    if (it.next() < 0)
        it.remove();
```

Choose one (90 seconds)

- **A.** Version 1 only
- **B.** Version 2 only
- **C.** Versions 2 and 3
- **D.** All three

Hint: all three compile and run (with `import java.util.*;`). `i` is an `int`, so `xs.remove(i)` removes by index. Trace each version on `[4, -1, -2, 5]`.

Q2 Multiple choice Which loop removes every negative?

Goal: remove **every** negative number from the `List<Integer> xs`, which starts as `[4, -1, -2, 5]`.

Version 1

```
for (int i = 0;
     i < xs.size(); i++)
    if (xs.get(i) < 0)
        xs.remove(i);
```

Version 2

```
for (int i = xs.size() - 1;
     i >= 0; i--)
    if (xs.get(i) < 0)
        xs.remove(i);
```

Version 3

```
Iterator<Integer> it
    = xs.iterator();
while (it.hasNext())
    if (it.next() < 0)
        it.remove();
```

Choose one (90 seconds)

- **A.** Version 1 only
- **B.** Version 2 only
- ✓ **C.** Versions 2 and 3
- **D.** All three

Answer: C

V1 ✗ ends `[4, -2, 5]`: once -1 goes, -2 slides into index 1 and `i++` skips it.

V2 ✓ backwards: a removal shifts only elements already checked.

V3 ✓ `it.remove()` removes what `next()` just gave.

A and B each miss a version that works; D includes V1.

Q3 Multiple choice Which line does not compile?

```
class Box<T> {
    private T value;
    Box(T value) { this.value = value; }
    T get() { return value; }
    void set(T value) { this.value = value; }
}
```

Choose one (60 seconds)

A. `int n = b.get() + 1;`

B. `b.set(7);`

C. `b.set("7");`

D. `Integer m = b.get();`

`main` is in another class, `Demo`, in the same file, and starts with:

```
Box<Integer> b = new Box<Integer>(5);
```

Each option is the next line of `main`.
Exactly **one** of them does not compile.

Q3 Multiple choice Which line does not compile?

```
class Box<T> {
    private T value;
    Box(T value) { this.value = value; }
    T get() { return value; }
    void set(T value) { this.value = value; }
}
```

Choose one (60 seconds)

A. `int n = b.get() + 1;`

B. `b.set(7);`

✓ **C.** `b.set("7");`

D. `Integer m = b.get();`

`main` is in another class, `Demo`, in the same file, and starts with:

```
Box<Integer> b = new Box<Integer>(5);
```

Answer: C

In `b`, `T` is `Integer`, so `set` needs an `Integer`:

`incompatible types: String cannot be converted to Integer`

A: `get()` gives an `Integer`; Java unboxes it to add 1.

B: the `int` 7 is autoboxed to an `Integer`.

D: `get()` returns `T`, which is `Integer` here.

Q4 Predict the output Insert, set and remove

```
1  // in main (import java.util.*;)
2  List<Integer> xs = new ArrayList<Integer>();
3  xs.add(10);
4  xs.add(20);
5  xs.add(0, 30);
6  xs.set(1, 20);
7  xs.add(xs.size(), 2);
8  System.out.println(xs + " " + xs.size());
9  xs.remove(2);
10 System.out.println(xs);
11 xs.remove(Integer.valueOf(2));
12 System.out.println(xs + " " + xs.get(1));
```

Predict (90 seconds, alone)

1. Write down the **three** lines that are printed.

2. Line 7, `xs.add(xs.size(), 2)`, is allowed. Would `xs.get(xs.size())` be allowed after line 12? Why?

Q4 Predict the output Insert, set and remove

```
1  // in main (import java.util.*;)
2  List<Integer> xs = new ArrayList<Integer>();
3  xs.add(10);
4  xs.add(20);
5  xs.add(0, 30);
6  xs.set(1, 20);
7  xs.add(xs.size(), 2);
8  System.out.println(xs + " " + xs.size());
9  xs.remove(2);
10 System.out.println(xs);
11 xs.remove(Integer.valueOf(2));
12 System.out.println(xs + " " + xs.get(1));
```

Answers

1. `[30, 20, 20, 2] 4`
`[30, 20, 2]`
`[30, 20] 20`

Line 5 **inserts** 30 at the front; line 6 **replaces** the 10; line 7 adds at the end. On line 9, 2 is an `int`, so an **index**: the second 20 goes. Line 11 removes the **value** 2.

2. No: `get` needs 0 to `size()`-1. `add(size(), x)` means "at the end"; `get(size())` is past it:
`IndexOutOfBoundsException: Index 2 out of bounds for length 2`

Q5 Predict the output A chain of nodes

```
1  class Node:
2      def __init__(self, value, link=None):
3          self.value = value
4          self.next = link
5
6  c = Node("C")
7  a = Node("A", Node("B", c))
8  a.next.next = Node("X", c)
9  a.next = a.next.next
10 names = []
11 n = a
12 while n is not None:
13     names.append(n.value)
14     n = n.next
15 print(names)
16 print(a.next.value, c.next)
```

Predict (90 seconds, alone)

1. Write down the **two** lines that are printed.

2. After line 9, which node can no longer be reached from `a`? Did any value have to move?

Tip: draw each node as a box with an arrow for `next`.

Q5 Predict the output A chain of nodes

```
1  class Node:
2      def __init__(self, value, link=None):
3          self.value = value
4          self.next = link
5
6  c = Node("C")
7  a = Node("A", Node("B", c))
8  a.next.next = Node("X", c)
9  a.next = a.next.next
10 names = []
11 n = a
12 while n is not None:
13     names.append(n.value)
14     n = n.next
15 print(names)
16 print(a.next.value, c.next)
```

Answers

1. `['A', 'X', 'C']`
`X None`

Line 7 builds A → B → C. Line 8 changes B's link to a new node X that links to C: A → B → X → C. Line 9 sets A's link to B's link (X): A → X → C. C is the last node, so `c.next` is `None`.

2. B: no link leads to it any more. No value moved; only links changed. That is why a linked list can insert and remove without shifting.

Q6 Find the bug The mark book (Java)

```
1  import java.util.ArrayList;
2  public class MarkBook {
3      private ArrayList<Integer> marks
4          = new ArrayList<Integer>();
5      void add(int m) { marks.add(0, m); }
6      void change(int i, int m) {
7          marks.add(i, m);
8      }
9      boolean removeMark(int m) {
10         return marks.remove(m);
11     }
12     int last() {
13         return marks.get(marks.size());
14     }
15     public String toString() { return "" + marks; }
16 }
```

✗ **Find 4 bugs** (3 min): 1 compile error, 3 logic errors (one of them crashes). Give each line number and its fix.

What the class should do

- `add(m)` adds `m` at the **end**
- `change(i, m)` replaces index `i`
- `removeMark(m)` removes one **value** `m`; if there is none, returns `false`
- `last()` gives the last mark

Test (in `main`)

```
MarkBook b = new MarkBook();
b.add(70); b.add(40); b.add(90);
b.change(1, 45);
System.out.println(b.removeMark(90) + " "
    + b.removeMark(12));
System.out.println(b + " " + b.last());
```

Expected output

```
true false
[70, 45] 45
```

Q6 Find the bug The mark book (Java)

```
1  import java.util.ArrayList;
2  public class MarkBook {
3      private ArrayList<Integer> marks
4          = new ArrayList<Integer>();
5      void add(int m) { marks.add(0, m); }
6      void change(int i, int m) {
7          marks.add(i, m);
8      }
9      boolean removeMark(int m) {
10         return marks.remove(m);
11     }
12     int last() {
13         return marks.get(marks.size());
14     }
15     public String toString() { return "" + marks; }
16 }
```

✗ **Find 4 bugs** (3 min): 1 compile error, 3 logic errors (one of them crashes). Give each line number and its fix.

✓ The 4 fixes

- **Line 10** (compile error): `m` is an `int`, so this is remove-at-**index**, which returns an `Integer`: `Integer cannot be converted to boolean`. Use `marks.remove(Integer.valueOf(m))`.
- **Line 5**: `add(0, m)` inserts at the front. Use `marks.add(m)`.
- **Line 7**: `add(i, m)` inserts, so the list grows. Use `marks.set(i, m)`.
- **Line 13**: index `size()` is past the end. Use `marks.size() - 1`.

Fix only line 10: it prints `true false`, then crashes with `IndexOutOfBoundsException` at line 13.

Q7 Find the bug The basket (Python)

```
1  class Basket:
2      def __init__(self, items):
3          self.items = items
4      def take_out(self, item):
5          if item in self.items:
6              self.items.pop(item)
7      def remove_all(self, item):
8          for thing in self.items:
9              if thing == item:
10                 self.items.remove(thing)
11
12 shop = ["egg", "milk", "milk", "tea"]
13 b = Basket(shop)
14 b.remove_all("milk")
15 print(b.items)
16 b.take_out("egg")
17 print(b.items, shop)
```

✗ **Find 3 bugs** in lines 1–10 (3 min).
Give each line number and its fix.

What the class should do

- Each basket keeps its **own copy** of the list it is given
- `take_out(item)`: remove one `item`, if it is there
- `remove_all(item)`: remove **every** copy of `item`

Expected output

```
['egg', 'tea']
['tea'] ['egg', 'milk', 'milk', 'tea']
```

✗ Actual output (crashes at line 6)

```
['egg', 'milk', 'tea']
TypeError: 'str' object cannot be
interpreted as an integer
```

Q7 Find the bug The basket (Python)

```python
class Basket:
    def __init__(self, items):
        self.items = items
    def take_out(self, item):
        if item in self.items:
            self.items.pop(item)
    def remove_all(self, item):
        for thing in self.items:
            if thing == item:
                self.items.remove(thing)

shop = ["egg", "milk", "milk", "tea"]
b = Basket(shop)
b.remove_all("milk")
print(b.items)
b.take_out("egg")
print(b.items, shop)
```

✗ **Find 3 bugs** in lines 1–10 (3 min).
Give each line number and its fix.

✓ The 3 fixes

- **Line 3**: shares the caller's list, so `shop` changes too. Copy it: `list(items)`.
- **Line 6**: pop takes an **index**, so `pop("egg")` crashes. By **value**: `self.items.remove(item)`.
- **Lines 8–10**: removing while looping: the 2nd `"milk"` slides into the place just checked and is skipped. Build a new list: `self.items = [t for t in self.items if t != item]`.

Q8 Write the code The last few, for any type (Java)

Finish the TODOs (alone, 4 min). A `Recent<T>` keeps only the last `max` items added.

- `add(item)`: add at the end; if there are now more than `max` items, remove the oldest (index 0).
- `latest()`: return the item added last, or `null` if there is none.

Skeleton

```
class Recent<T> {
    private final int max;
    private final ArrayList<T> items
        = new ArrayList<T>();
    Recent(int max) { this.max = max; }
    void add(T item) { /* TODO */ }
    T latest() {
        return null;  // TODO
    }
    public String toString() {
        return items.toString();
    }
}
```

Test (in `main`)

```
Recent<String> r = new Recent<String>(3);
r.add("tea"); r.add("milk");
r.add("jam"); r.add("egg");
System.out.println(r + " " + r.latest());
Recent<Integer> n = new Recent<Integer>(2);
System.out.println(n.latest());
n.add(7); n.add(8); n.add(9);
System.out.println(n + " " + n.latest());
```

Must print

```
[milk, jam, egg] egg
null
[8, 9] 9
```

Q8 Write the code The last few, for any type (Java)

Finish the TODOs (alone, 4 min). A `Recent<T>` keeps only the last `max` items added.

- `add(item)`: add at the end; if there are now more than `max` items, remove the oldest (index 0).
- `latest()`: return the item added last, or `null` if there is none.

Skeleton

```
class Recent<T> {
    private final int max;
    private final ArrayList<T> items
        = new ArrayList<T>();
    Recent(int max) { this.max = max; }
    void add(T item) { /* TODO */ }
    T latest() {
        return null;  // TODO
    }
    public String toString() {
        return items.toString();
    }
}
```

✓ One answer (the two TODO methods)

```
void add(T item) {
    items.add(item);
    if (items.size() > max)
        items.remove(0);
}
T latest() {
    if (items.isEmpty()) return null;
    return items.get(items.size() - 1);
}
```

`remove(0)` removes by **index** even in `Recent<Integer>`: 0 is an `int`. That is why the test gives `[8, 9]`.

Q9 Write the code A help-desk queue (Python)

Finish the TODOs (alone, 4 min). First come, first served. Use the deque: append adds at the back, popleft removes from the front.

- `join(name)`: add `name` at the back; return how many are now waiting.
- `serve()`: remove and return the **front** name, or `None` if nobody is waiting.

Skeleton

```
from collections import deque

class HelpDesk:
    def __init__(self):
        self.waiting = deque()
    def join(self, name):
        return 0  # TODO
    def serve(self):
        return None  # TODO
    def size(self):
        return len(self.waiting)
```

Test

```
q = HelpDesk()
print(q.join("An"), q.join("Binh"), q.join("Chi"))
print(q.serve(), q.serve())
print(q.join("Dung"), q.size())
print(q.serve(), q.serve(), q.serve())
```

Must print

```
1 2 3
An Binh
2 2
Chi Dung None
```

Q9 Write the code A help-desk queue (Python)

Finish the `TODO`s (alone, 4 min). First come, first served. Use the `deque`: `append` adds at the back, `popleft` removes from the front.

- `join(name)`: add `name` at the back; return how many are now waiting.
- `serve()`: remove and return the **front** name, or `None` if nobody is waiting.

Skeleton

```
from collections import deque

class HelpDesk:
    def __init__(self):
        self.waiting = deque()
    def join(self, name):
        return 0  # TODO
    def serve(self):
        return None  # TODO
    def size(self):
        return len(self.waiting)
```

✓ One answer (the two `TODO` methods)

```
def join(self, name):
    self.waiting.append(name)
    return len(self.waiting)
def serve(self):
    if len(self.waiting) == 0:
        return None
    return self.waiting.popleft()
```

Why not a list and `pop(0)`? It works, but every `pop(0)` shifts all the others left. `popleft` does not.

Q10 Write the code A queue of linked nodes (Java)

Finish the `TODO`s (alone, 4 min). `head` is the front node, `tail` the back.

- `dequeue()`: remove the front node and return its value. Empty queue: `tail` is `null` too.
- `size()`: count the nodes by following `next` from `head`.

Skeleton

```
class Node {
    int value; Node next;
    Node(int v) { value = v; }
}
class IntQueue {
    private Node head, tail;
    void enqueue(int v) {
        Node n = new Node(v);
        if (tail == null) head = n;
        else tail.next = n;
        tail = n;
    }
    int dequeue() { return 0; }  // TODO
    int size() { return 0; }     // TODO
}
```

Test (in `main`)

```
IntQueue q = new IntQueue();
q.enqueue(5); q.enqueue(6); q.enqueue(7);
System.out.println(q.dequeue() + " " + q.size());
System.out.println(q.dequeue() + " "
    + q.dequeue() + " " + q.size());
q.enqueue(8);
System.out.println(q.dequeue() + " " + q.size());
```

Must print

```
5 2
6 7 0
8 0
```

Q10 Write the code A queue of linked nodes (Java)

Finish the `TODO`s (alone, 4 min). `head` is the front node, `tail` the back.

- `dequeue()`: remove the front node and return its value. Empty queue: `tail` is `null` too.
- `size()`: count the nodes by following `next` from `head`.

Skeleton

```
class Node {
    int value; Node next;
    Node(int v) { value = v; }
}
class IntQueue {
    private Node head, tail;
    void enqueue(int v) {
        Node n = new Node(v);
        if (tail == null) head = n;
        else tail.next = n;
        tail = n;
    }
    int dequeue() { return 0; }  // TODO
    int size() { return 0; }     // TODO
}
```

✓ One answer (the two `TODO` methods)

```
int dequeue() {
    int v = head.value;
    head = head.next;
    if (head == null) tail = null;
    return v;
}
int size() {
    int count = 0;
    for (Node n = head; n != null; n = n.next)
        count++;
    return count;
}
```

Without `tail = null`, the last test line crashes with a `NullPointerException`: 8 is linked behind the old 7.

Q11 Predict the output Splice the nodes (Java)

```java
class Node {
    int value; Node next;
    Node(int v, Node n) { value = v; next = n; }
}
// in main
Node head = new Node(1, new Node(2, new Node(3, null)));
head = new Node(0, head);
head.next.next = head.next.next.next;
Node p = head.next;
p.value = 5;
p = p.next;
p = new Node(7, null);
String s = "";
for (Node n = head; n != null; n = n.next)
    s += n.value + " ";
System.out.println(s);
System.out.println(p.value + " " + head.next.next.value);
```

Predict (2 minutes, alone)

1. Write down the **two** lines that are printed.

2. Line 10 changed the list, but line 12 did not add 7 to it. Why not?

Tip: draw each node as a box with an arrow for `next`, and redraw after every line.

Q11 Predict the output Splice the nodes (Java)

```java
class Node {
    int value; Node next;
    Node(int v, Node n) { value = v; next = n; }
}
// in main
Node head = new Node(1, new Node(2, new Node(3, null)));
head = new Node(0, head);
head.next.next = head.next.next.next;
Node p = head.next;
p.value = 5;
p = p.next;
p = new Node(7, null);
String s = "";
for (Node n = head; n != null; n = n.next)
    s += n.value + " ";
System.out.println(s);
System.out.println(p.value + " " + head.next.next.value);
```

Answers

1. 0 5 3
7 3

Line 6: $1 \to 2 \to 3$. Line 7 puts a new 0 in front: $0 \to 1 \to 2 \to 3$. Line 8 links node 1 past node 2: $0 \to 1 \to 3$. Line 10 changes the value **inside** node 1: $0 \to 5 \to 3$.

2. `p` only points at a node. Line 10 changes **that node**, so the list sees it. Line 12 points `p` at a **new** node that no `next` leads to. To add 7 at the end, write `p.next = new Node(7, null);` instead.

Q12 Predict the output Who holds the new head? (Python)

```python
class Node:
    def __init__(self, value, next=None):
        self.value = value
        self.next = next
def show(head):
    out = []
    while head is not None:
        out.append(head.value)
        head = head.next
    return out
def push_front(head, value):
    return Node(value, head)
def remove_after(node):
    if node.next is not None:
        node.next = node.next.next
h = Node("b", Node("c"))
push_front(h, "z")
h = push_front(h, "a")
remove_after(h.next)
print(show(h))
print(h.value, h.next.next)
```

Predict (2 minutes, alone)

1. Write down the **two** lines that are printed.

2. Where did `"z"` go?

3. `show` moves `head` until it is `None`. Is `h` `None` after line 20?

Q12 Predict the output Who holds the new head? (Python)

```python
class Node:
    def __init__(self, value, next=None):
        self.value = value
        self.next = next
def show(head):
    out = []
    while head is not None:
        out.append(head.value)
        head = head.next
    return out
def push_front(head, value):
    return Node(value, head)
def remove_after(node):
    if node.next is not None:
        node.next = node.next.next
h = Node("b", Node("c"))
push_front(h, "z")
h = push_front(h, "a")
remove_after(h.next)
print(show(h))
print(h.value, h.next.next)
```

Answers

1. `['a', 'b']`
`a None`

Line 16: b → c. Line 18: a → b → c. Line 19 is called on `h.next`, node b, so it removes the node **after** b (c): a → b. b is now last, so `h.next.next` is `None`.

2. Line 17 makes a z node that links to b, but nobody keeps the returned node. Nothing leads to z, so it is lost. Line 18 keeps it: `h = push_front(…)`.

3. No. Inside `show`, `head` is a separate name for the same first node. Moving that name never changes `h`.

Q13 Find the bug The linked queue (Java)

```java
class Node {
    String value; Node next;
    Node(String v) { value = v; }
}
class LinkedQueue {
    private Node head, tail;
    void enqueue(String v) {
        Node n = new Node(v);
        if (tail == null) { head = n; tail = n; }
        else tail.next = n;
    }
    String dequeue() {
        String v = head.value;
        head = head.next;
        return v;
    }
    int size() {
        int count = 0;
        for (Node n = head; n.next != null; n = n.next)
            count++;
        return count;
    }
}
```

✗ **Find 3 bugs** (3 min). Give each line number and its fix.

Test (in `main`)

```java
LinkedQueue q = new LinkedQueue();
q.enqueue("An"); q.enqueue("Mai"); q.enqueue("Tuan
    ");
System.out.println(q.dequeue() + " " + q.size());
System.out.println(q.dequeue() + " " + q.dequeue()
    );
q.enqueue("Linh");
System.out.println(q.dequeue() + " " + q.size());
```

Expected output

```
An 2
Mai Tuan
Linh 0
```

✗ Actual output

```
An 0
java.lang.NullPointerException
```

Q13 Find the bug The linked queue (Java)

```java
class Node {
    String value; Node next;
    Node(String v) { value = v; }
}
class LinkedQueue {
    private Node head, tail;
    void enqueue(String v) {
        Node n = new Node(v);
        if (tail == null) { head = n; tail = n; }
        else tail.next = n;
    }
    String dequeue() {
        String v = head.value;
        head = head.next;
        return v;
    }
    int size() {
        int count = 0;
        for (Node n = head; n.next != null; n = n.next)
            count++;
        return count;
    }
}
```

✗ **Find 3 bugs** (3 min). Give each line number and its fix.

✓ The 3 fixes

- **Line 10**: `tail` never moves, so each new node replaces An's link (Mai is lost). Use `else { tail.next = n; tail = n; }`
- **After line 14**: when the last node leaves, `tail` still points at it, and the next `enqueue` links behind it. Add `if (head == null) tail = null;`
- **Line 19**: `n.next != null` stops one node early, and crashes on an empty queue. Use `n != null`.

Fix only line 10: it prints `An 1` and `Mai Tuan`, then crashes on the last line: `head` is still `null`.

Q14 Find the bug The chain (Python)

```
1  class Node:
2      def __init__(self, value):
3          self.value, self.next = value, None
4  class Chain:
5      def __init__(self):
6          self.head = None
7      def add_last(self, value):
8          node = Node(value)
9          if self.head is None:
10             self.head = node
11             return
12         cur = self.head
13         while cur is not None:
14             cur = cur.next
15         cur.next = node
16     def remove(self, value):
17         prev, cur = None, self.head
18         while cur is not None and cur.value != value:
19             prev, cur = cur, cur.next
20         if cur is not None:
21             prev.next = cur
22     # values() returns the values in order
```

✗ **Find 3 bugs** in lines 7–21 (3 min). Give each line number and its fix.

Test (`values()`, lines 22–27, is correct)

```
c = Chain()
for v in ["a", "b", "c", "d"]:
    c.add_last(v)
c.remove("c")
c.remove("a")
c.remove("z")
print(c.values())
```

Expected output

```
['b', 'd']
```

✗ **Actual output** (crashes at line 15, adding "b")

```
AttributeError: 'NoneType' object
has no attribute 'next'
```

Q14 Find the bug The chain (Python)

```
1  class Node:
2      def __init__(self, value):
3          self.value, self.next = value, None
4  class Chain:
5      def __init__(self):
6          self.head = None
7      def add_last(self, value):
8          node = Node(value)
9          if self.head is None:
10             self.head = node
11             return
12         cur = self.head
13         while cur is not None:
14             cur = cur.next
15         cur.next = node
16     def remove(self, value):
17         prev, cur = None, self.head
18         while cur is not None and cur.value != value:
19             prev, cur = cur, cur.next
20         if cur is not None:
21             prev.next = cur
22     # values() returns the values in order
```

✗ **Find 3 bugs** in lines 7–21 (3 min). Give each line number and its fix.

✓ The 3 fixes

- **Line 13**: the loop runs past the last node, so line 15 has no node to link from. Stop **at** it: `while cur.next is not None:`
- **Line 21**: `prev.next = cur` keeps the link it already had, so nothing is removed. Skip the node: `prev.next = cur.next`
- **Line 21 again**: if the value is in the first node, `prev` is `None`. Move the head instead: `if prev is None: self.head = cur.next`

Fix only line 13: `remove("c")` does nothing, then `remove("a")` crashes.

Looking ahead: one method, two kinds of list

This week: program to `List`

```
static int total(List<Integer> xs) {
    int sum = 0;
    for (int x : xs) sum += x;
    return sum;
}
// in main
List<Integer> a = new ArrayList<Integer>();
a.add(2); a.add(3);
List<Integer> b = new LinkedList<Integer>(a);
b.add(5);
System.out.println(total(a) + " " + total(b));
```

Think (1 minute)

This prints `5 10`. `ArrayList` and `LinkedList` are different classes. Why can `total` accept **both**?

Looking ahead: one method, two kinds of list

This week: program to `List`

```
static int total(List<Integer> xs) {
    int sum = 0;
    for (int x : xs) sum += x;
    return sum;
}
// in main
List<Integer> a = new ArrayList<Integer>();
a.add(2); a.add(3);
List<Integer> b = new LinkedList<Integer>(a);
b.add(5);
System.out.println(total(a) + " " + total(b));
```

Think (1 minute)

This prints `5 10`. `ArrayList` and `LinkedList` are different classes. Why can `total` accept **both**?

`List` is an **interface**: methods (`add`, `get`, `size`, …) that a class promises to have. It stores nothing itself.

`ArrayList` (a backing array) and `LinkedList` (linked nodes) both **implement** `List`.

`total` uses only what `List` promises, so it works with both.

Next: Week 8

Write your **own** interface, let several classes keep it, reuse code with **inheritance**, and trace which method runs.

You should now be able to...

Lists and indexes

- ✓ insert, replace and remove at indexes 0 to `size()−1` (Q1, Q4, Q6)
- ✓ tell remove-by-index from remove-by-value, in Java and Python (Q4, Q6, Q7)
- ✓ remove while looping, safely (Q2, Q7)
- ✓ copy a list when an object needs its own (Q7)

Nodes, queues and generics

- ✓ trace and change the links between nodes (Q5, Q10–Q12)
- ✓ tell changing a node from re-pointing a variable (Q11, Q12)
- ✓ build and debug a first-in, first-out queue (Q9, Q10, Q13)
- ✓ add and remove at the ends of a chain, including the head (Q14)
- ✓ write and use a generic class like `Box<T>` (Q3, Q8)
- ✓ use `Integer`, not `int`, as a type argument (Q1, Q3)

Next: Week 8 – Inheritance, interfaces and polymorphism

Describe a shared contract, implement it in several classes, and trace which method runs.